

Deep Learning and Neural Networks

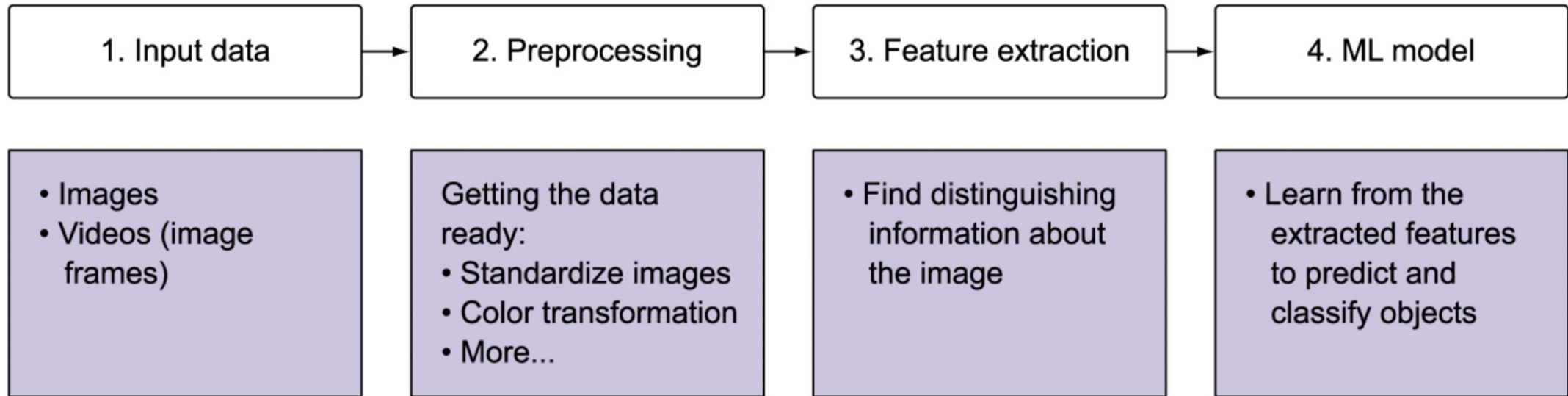
Priyanka Bagade, IITK

Lecture 3

Content Copyrights

- I own the copyright of the CS776: Deep Learning for Computer Vision course material. It includes lectures, presentations, exams and assignments. It should not be distributed in print or through electronic media without the consent of the instructor. Students can take notes or make their own copies of the content.

CV Pipeline



CV Pipeline using ML Model Example

Classifies the image and outputs the label or class (category)

1. Input data



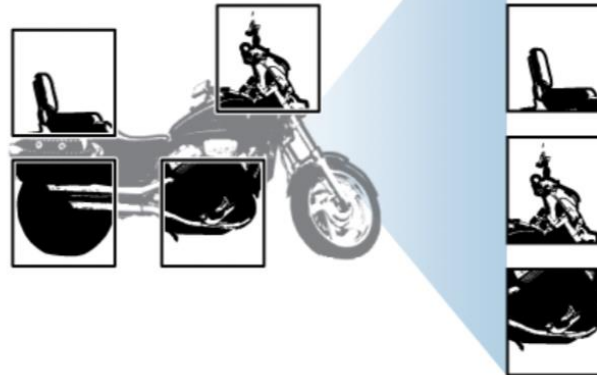
2. Preprocessing

- Geometric transforming
- Image blurring



e.g. standardizing the image, resizing, blurring, grayscale, rotating, changing shape etc

3. Feature extraction



Features vector

Extracting Unique Features

4. Classifier

$P(\text{motorcycle}) = 0.85$

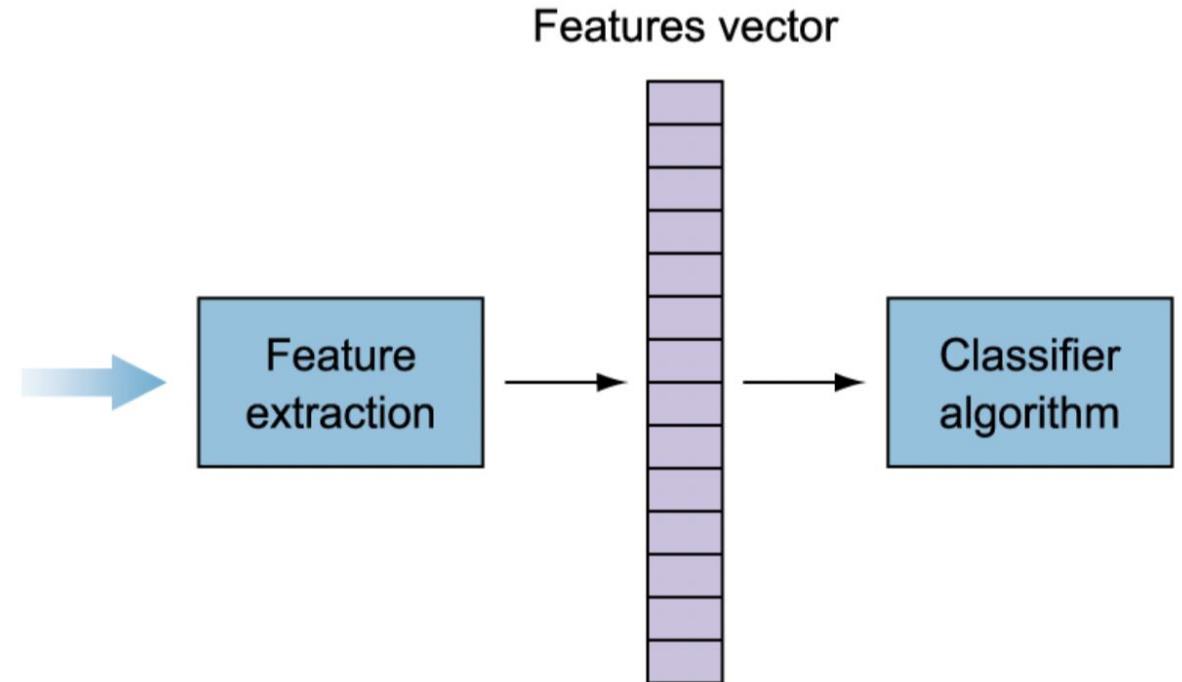
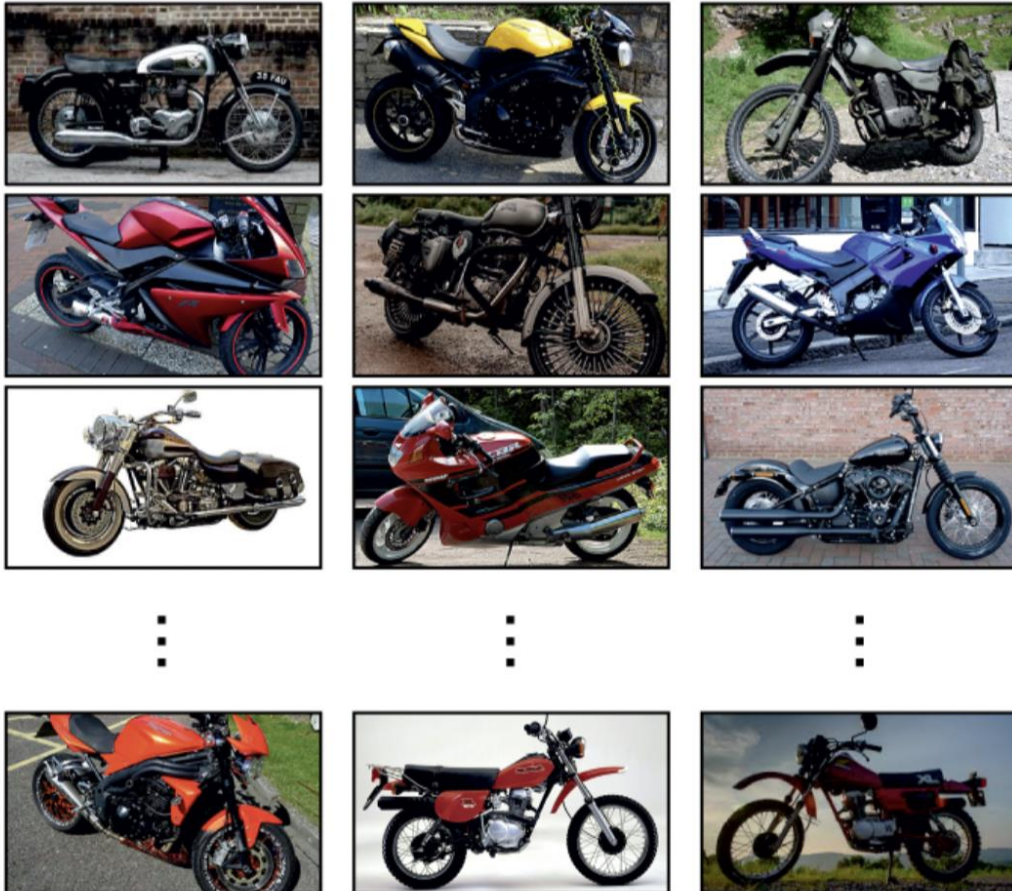
$P(\text{car}) = 0.14$

$P(\text{dog}) = 0.01$

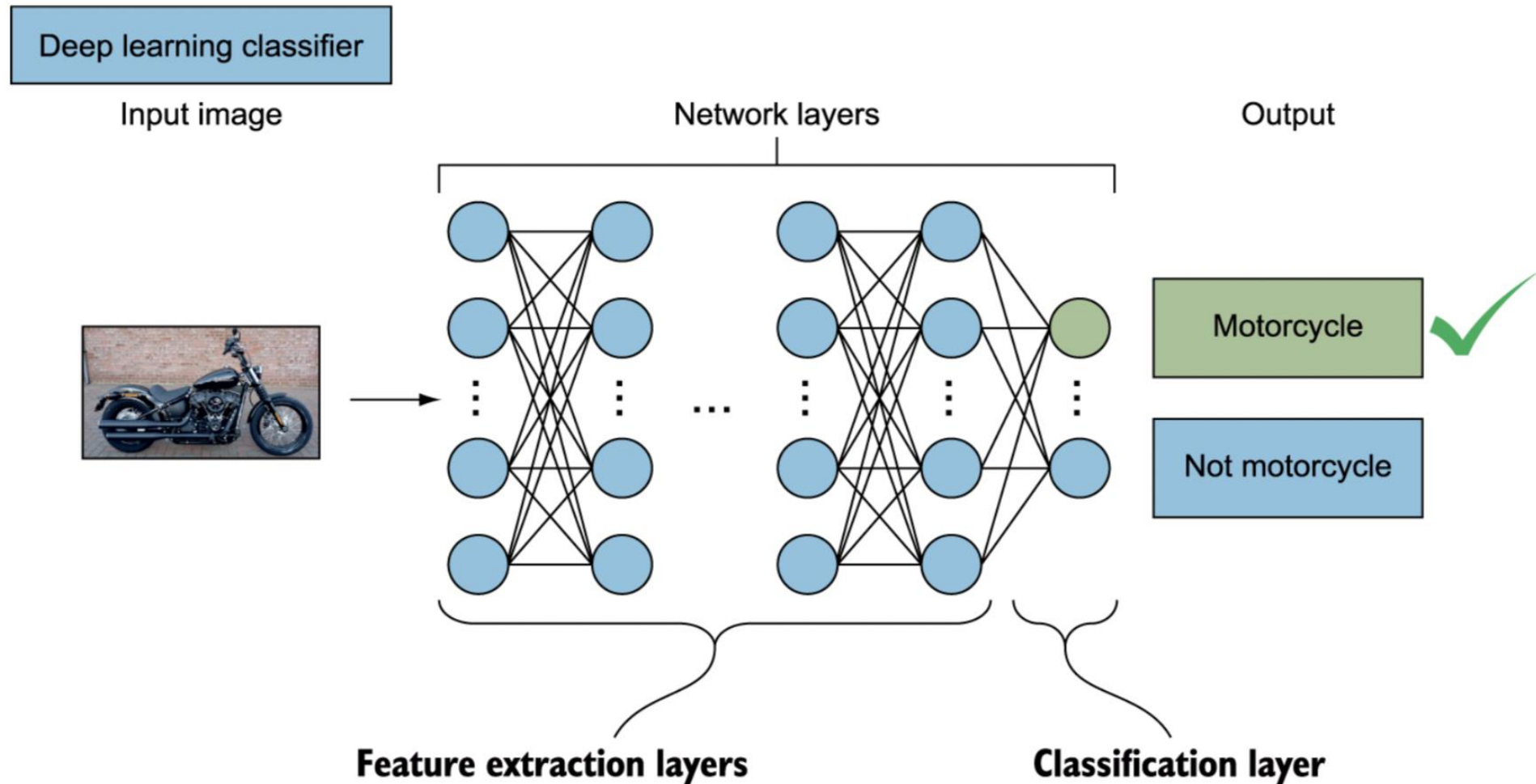


Extracting and Consolidating Features in a Vector

Images dataset of 10,000 images

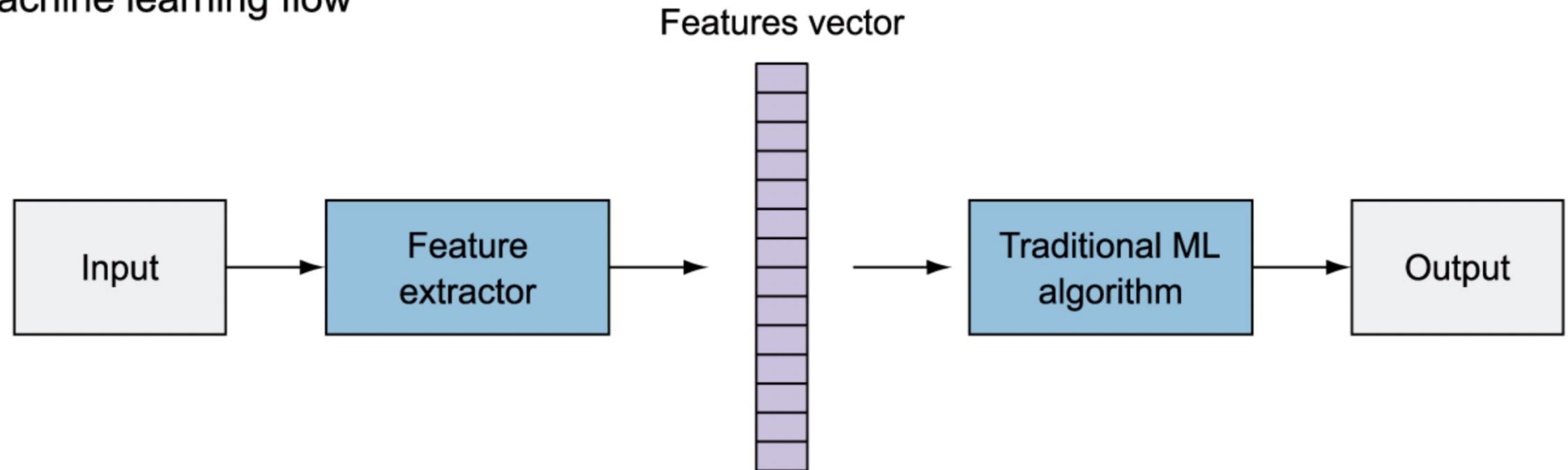


DL model to extract image features

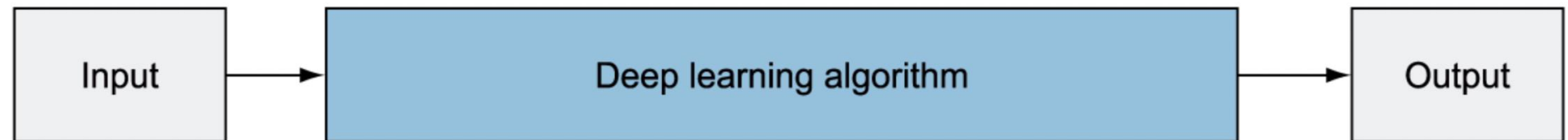


Learning Flow

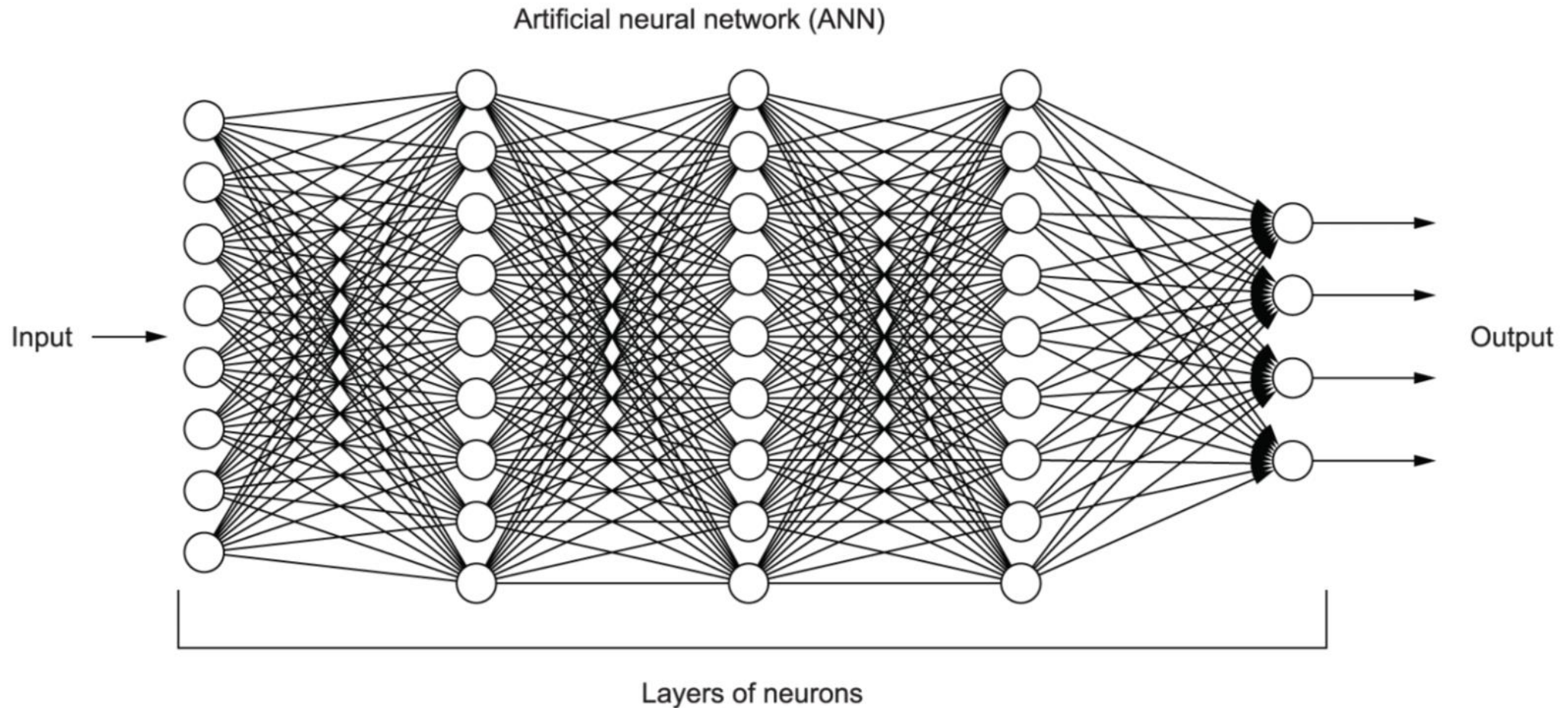
Traditional machine learning flow



Deep learning flow

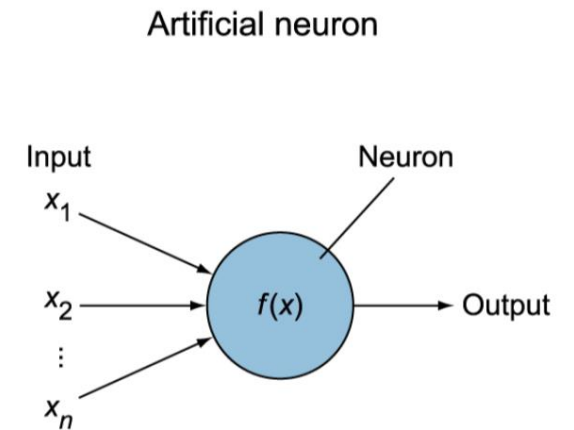
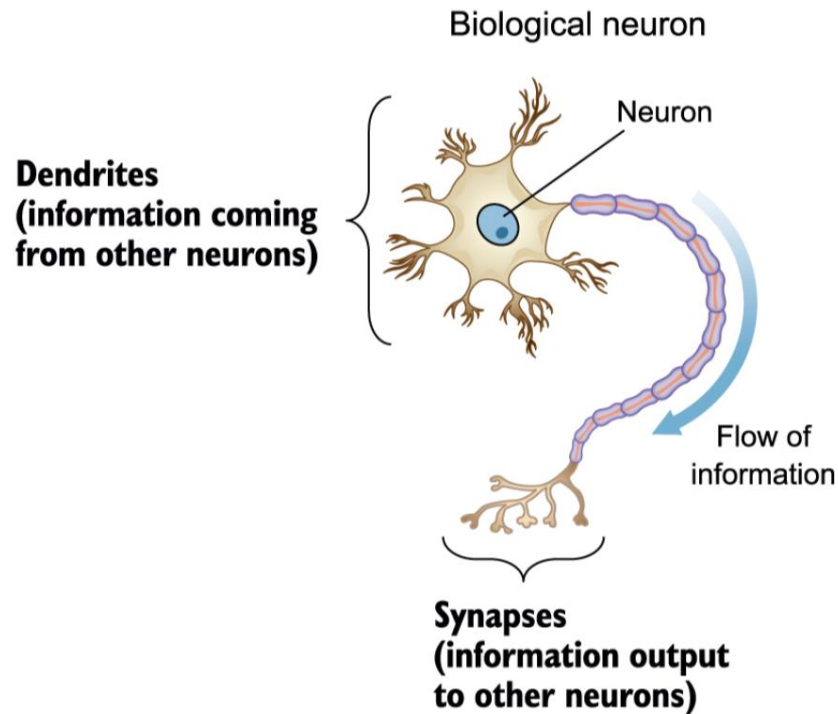


Artificial Neural Network (ANN) or Multilayer Perceptron (MLP)



Perceptron

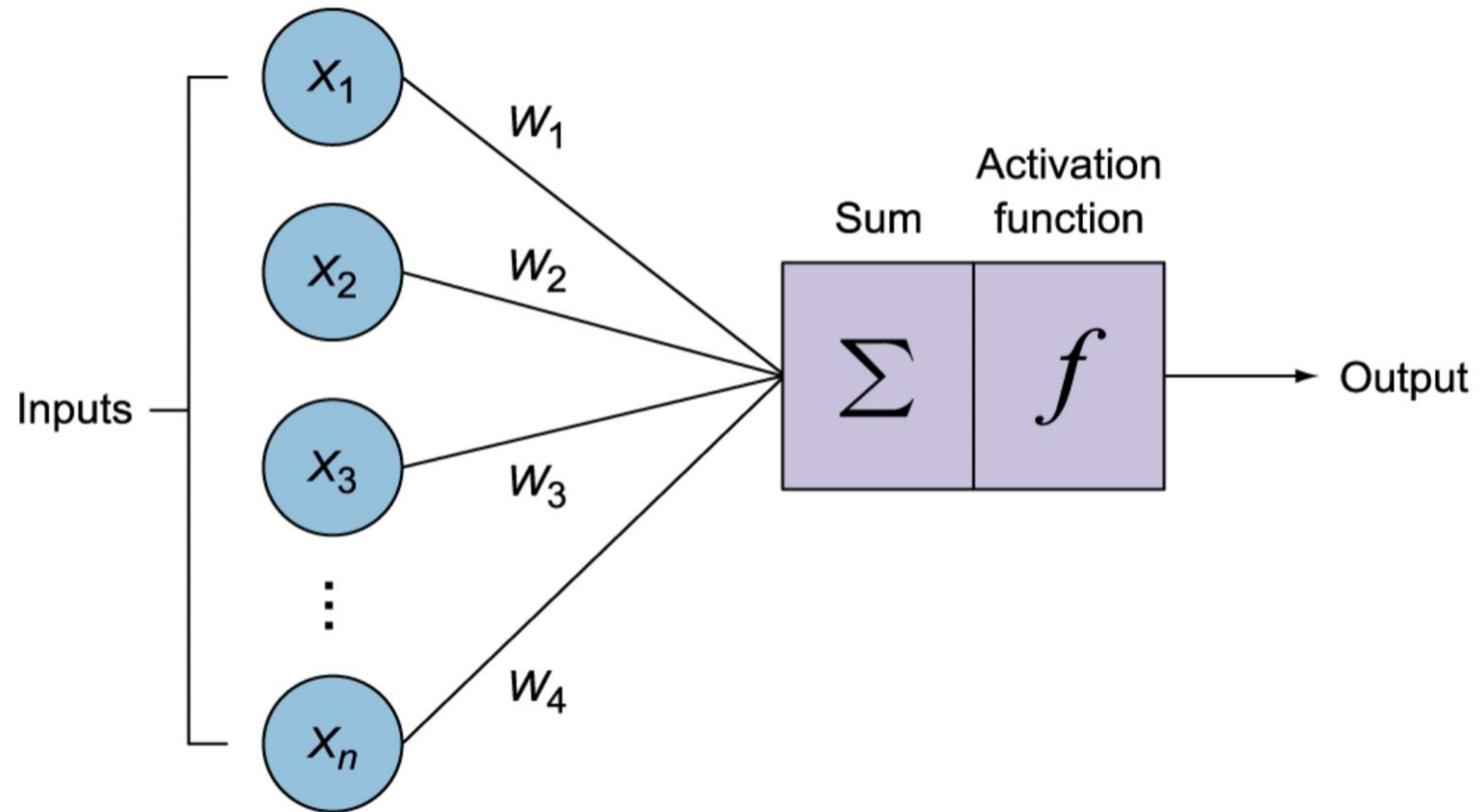
- The simplest neural network with a single neuron
- Similar to biological neuron
- Artificial neuron
 - Calculates weighted sum of the neurons
 - Uses a step function for the output to check threshold crossing condition
- Connection weights
 - Describes the effect of input feature on the output
 - Example: house price



Input = $x_1, x_2, \dots, x_n$, weights = $w_1, w_2, \dots, w_n$.

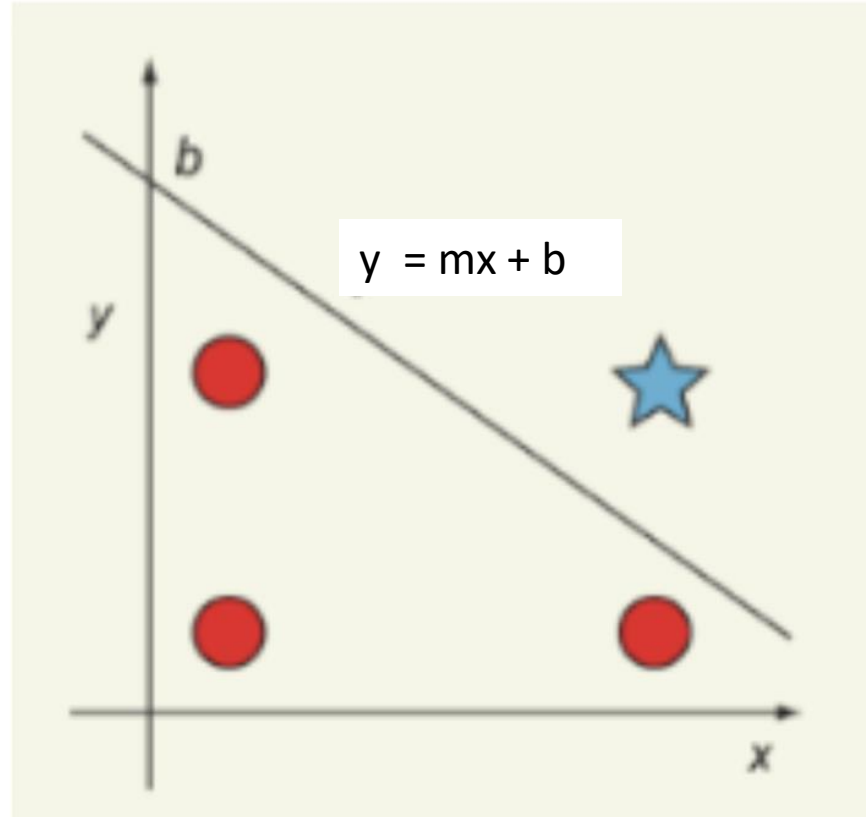
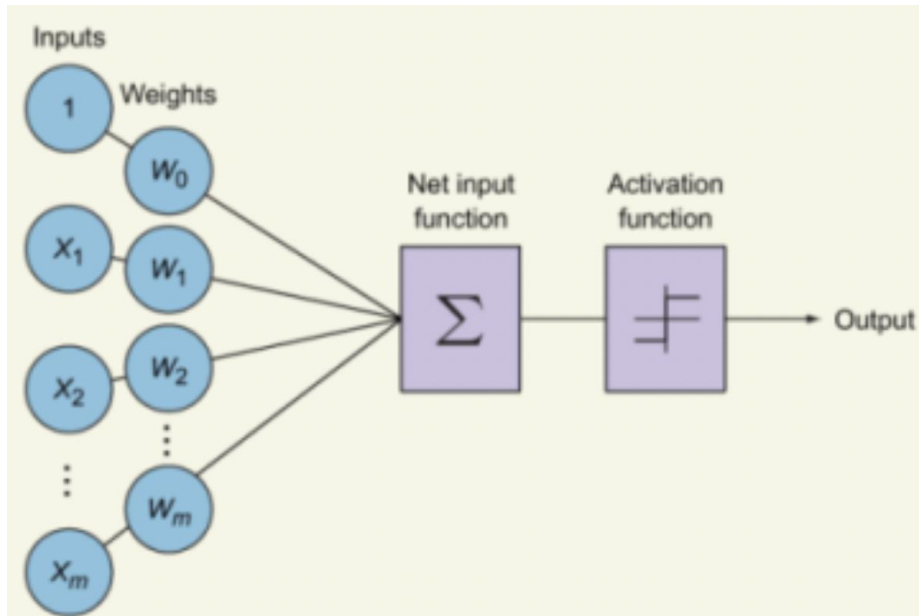
o/p = $\text{activation_function}(w_1.x_1 + w_2.x_2 + \dots + w_n.x_n)$

Perceptron

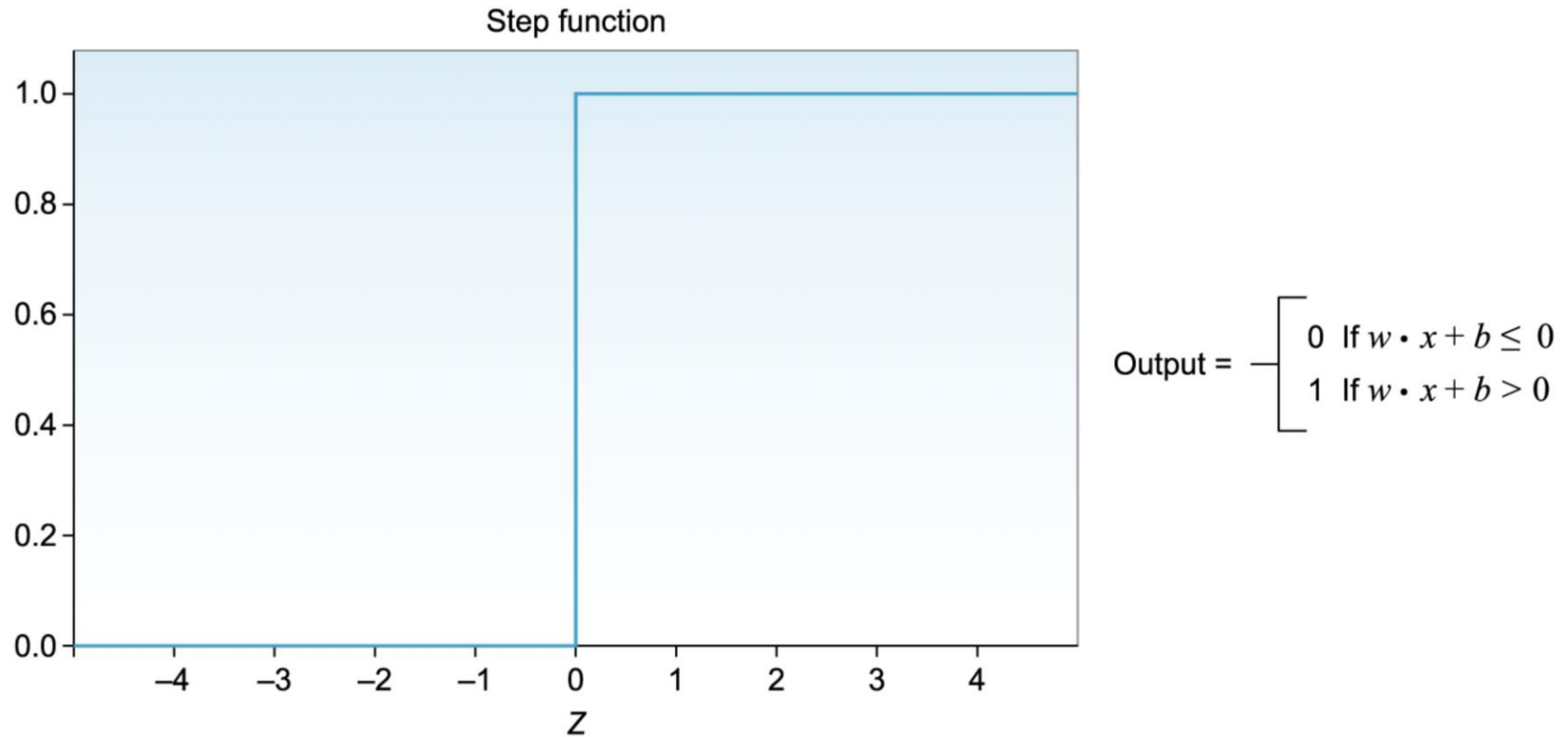


Bias in the Perceptron

- the y-intercept
- Considered as an extra weight
 - Learned and adjusted by the neuron



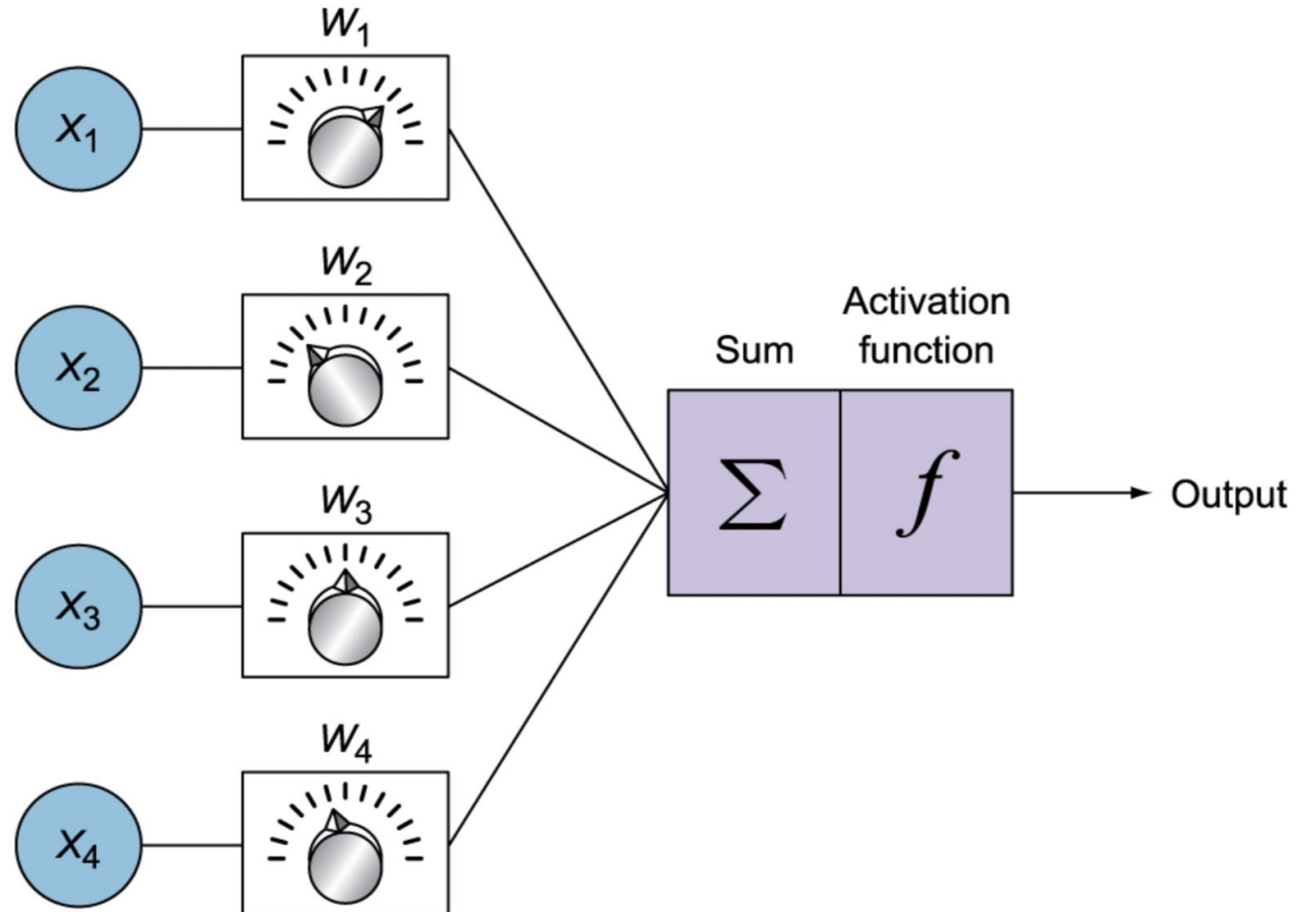
Step Activation Function



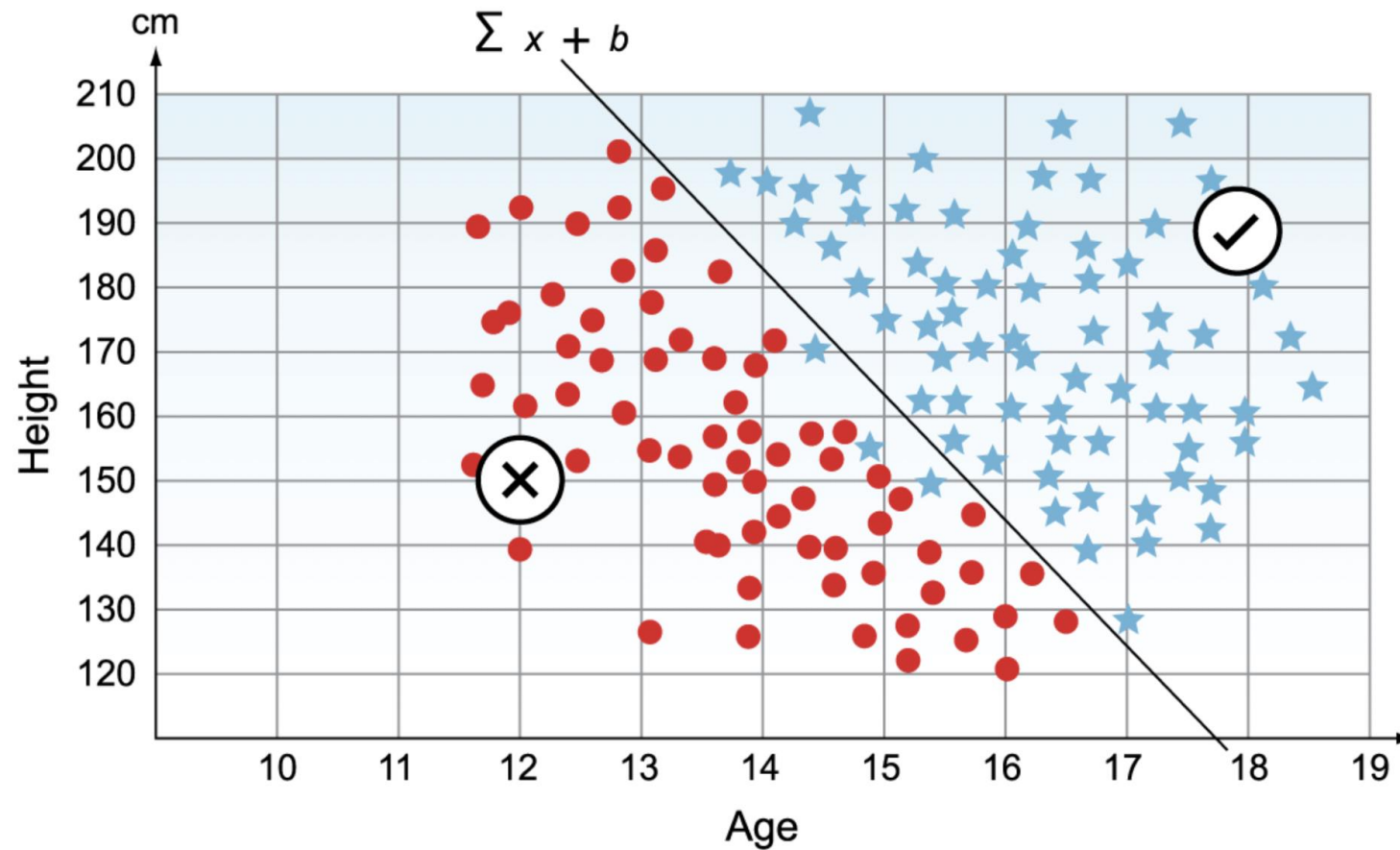
$\hat{y} = g(x)$, where g is an activation function and z is the weighted sum $= \sum x_i \cdot w_i + b$

How does the perceptron learn?

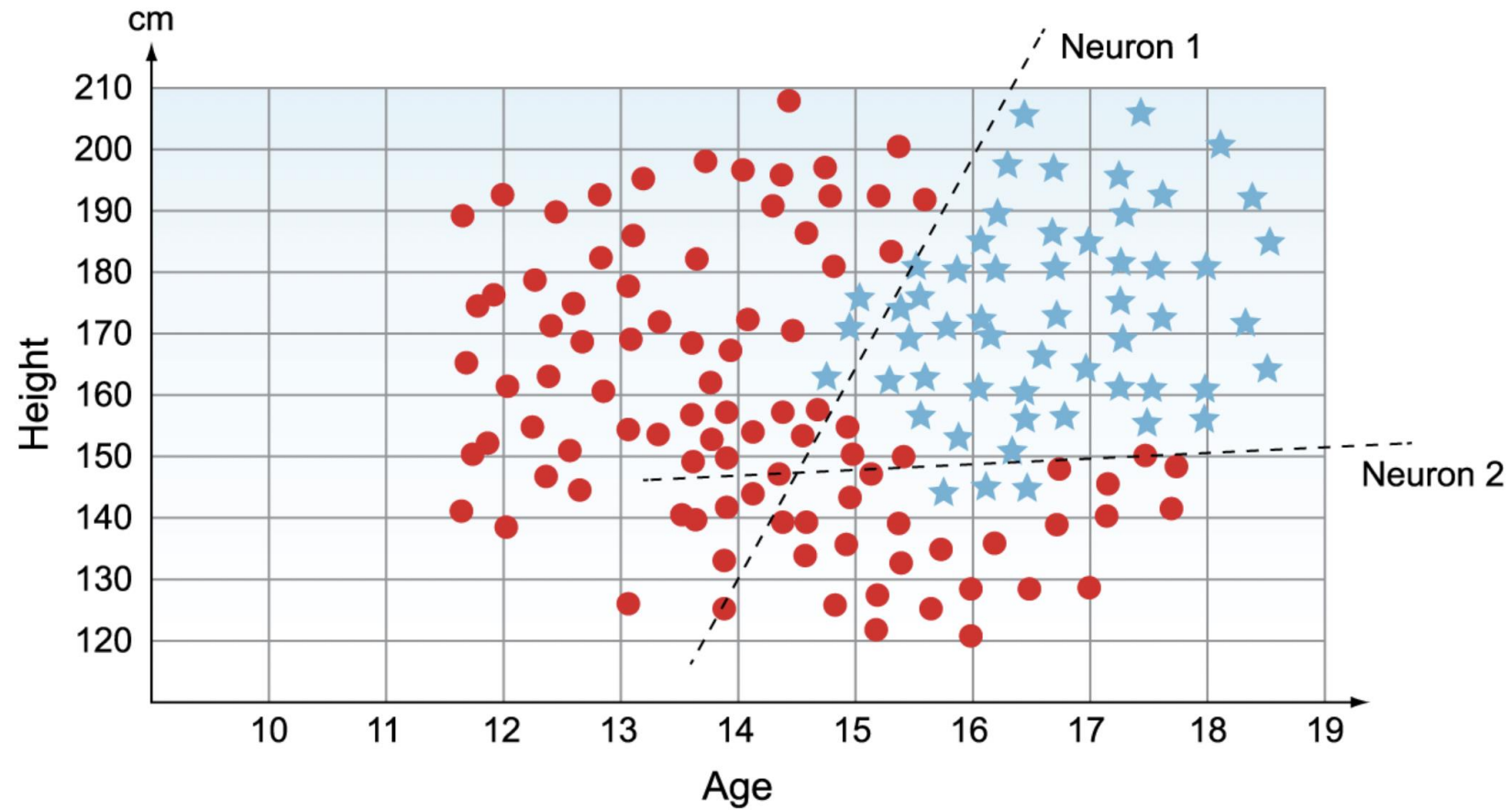
- Trial and error method
- Training algorithm
 - Neuron calculates the weighted sum and uses activation function to predict the output
 - Calculates the error by comparing the predicted output with the actual output
 - Updates the weights to reduce the error
 - Repeat!



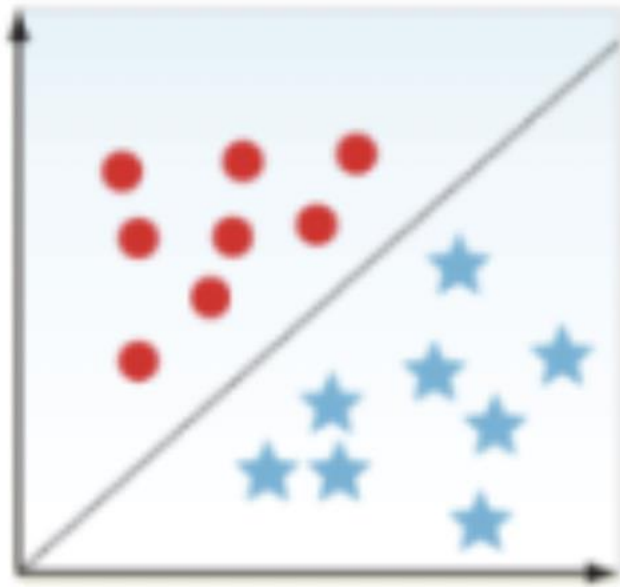
Is one neuron enough to solve complex problems?



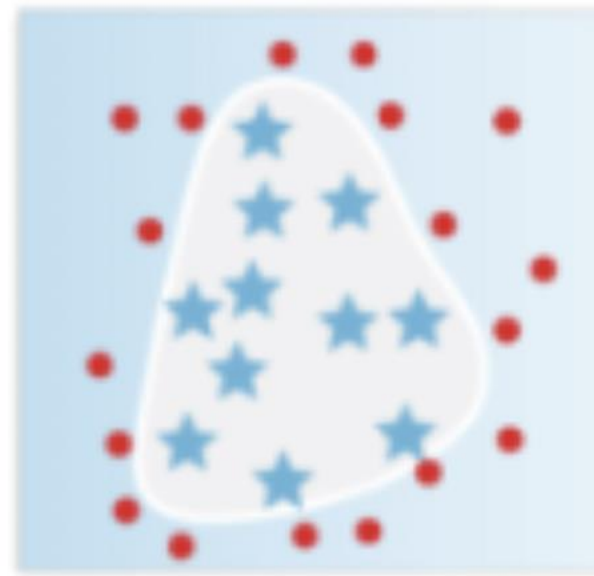
Non-linear dataset



Linear vs Non-linear datasets

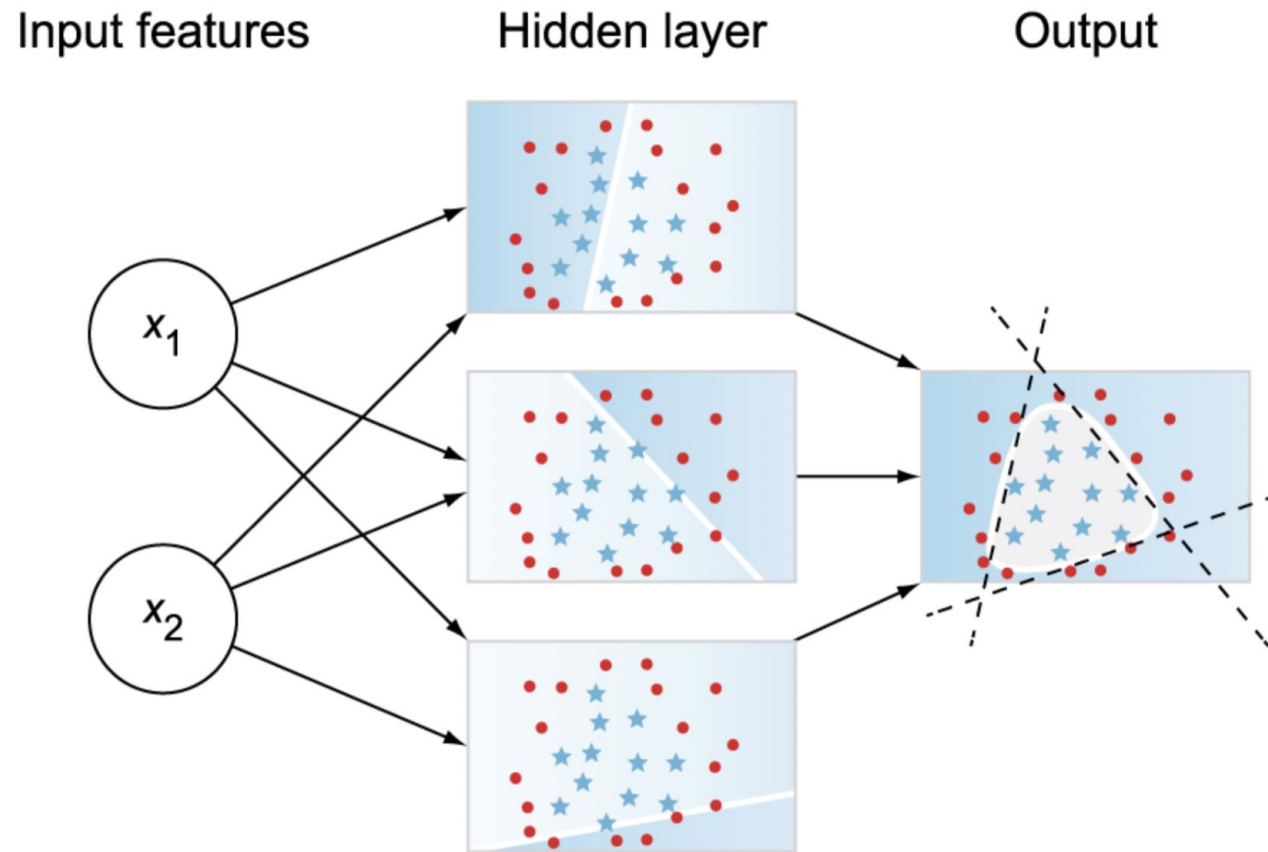


Linear

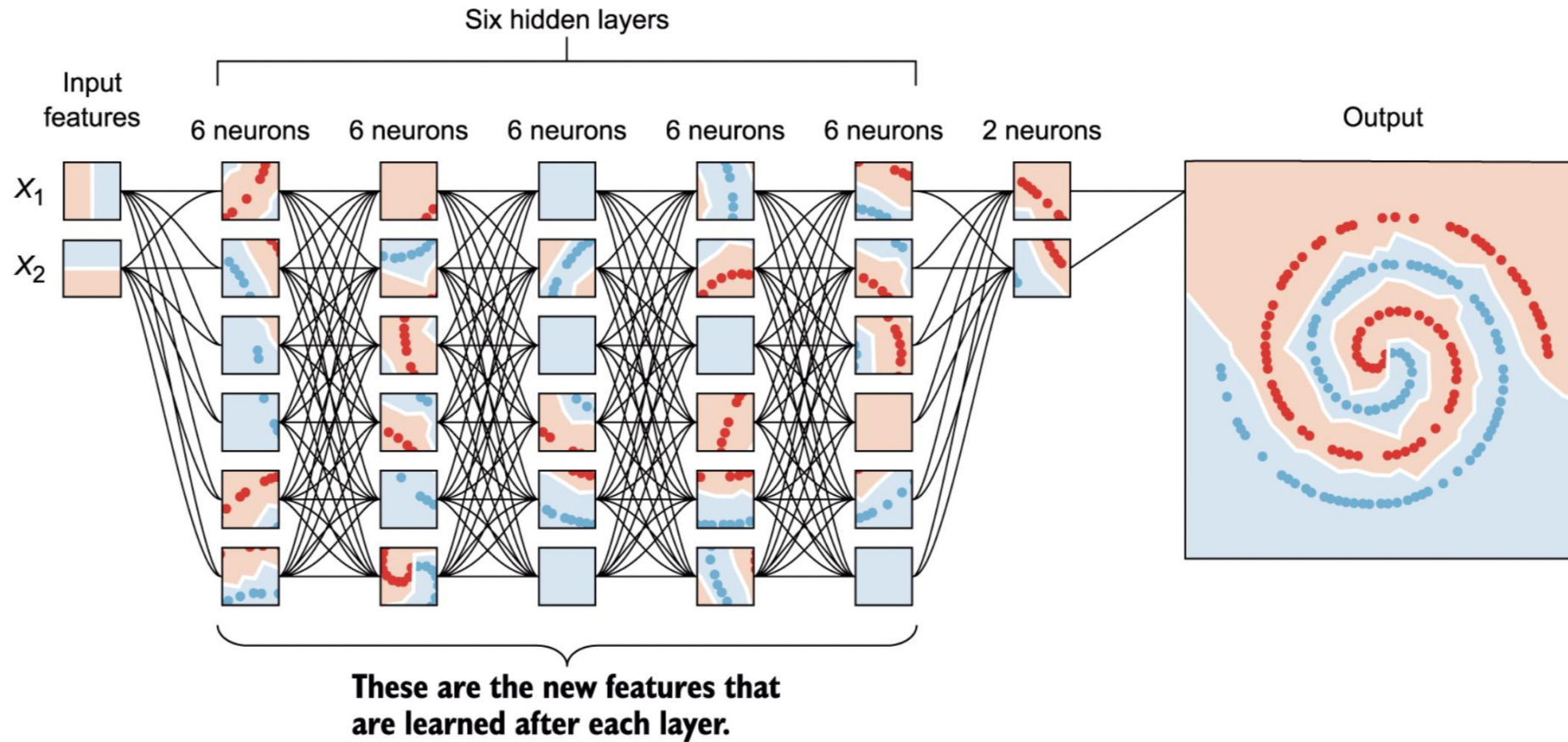


Non-linear

Multilayer Perceptron



Multilayer Perceptron Architecture



Input Layer

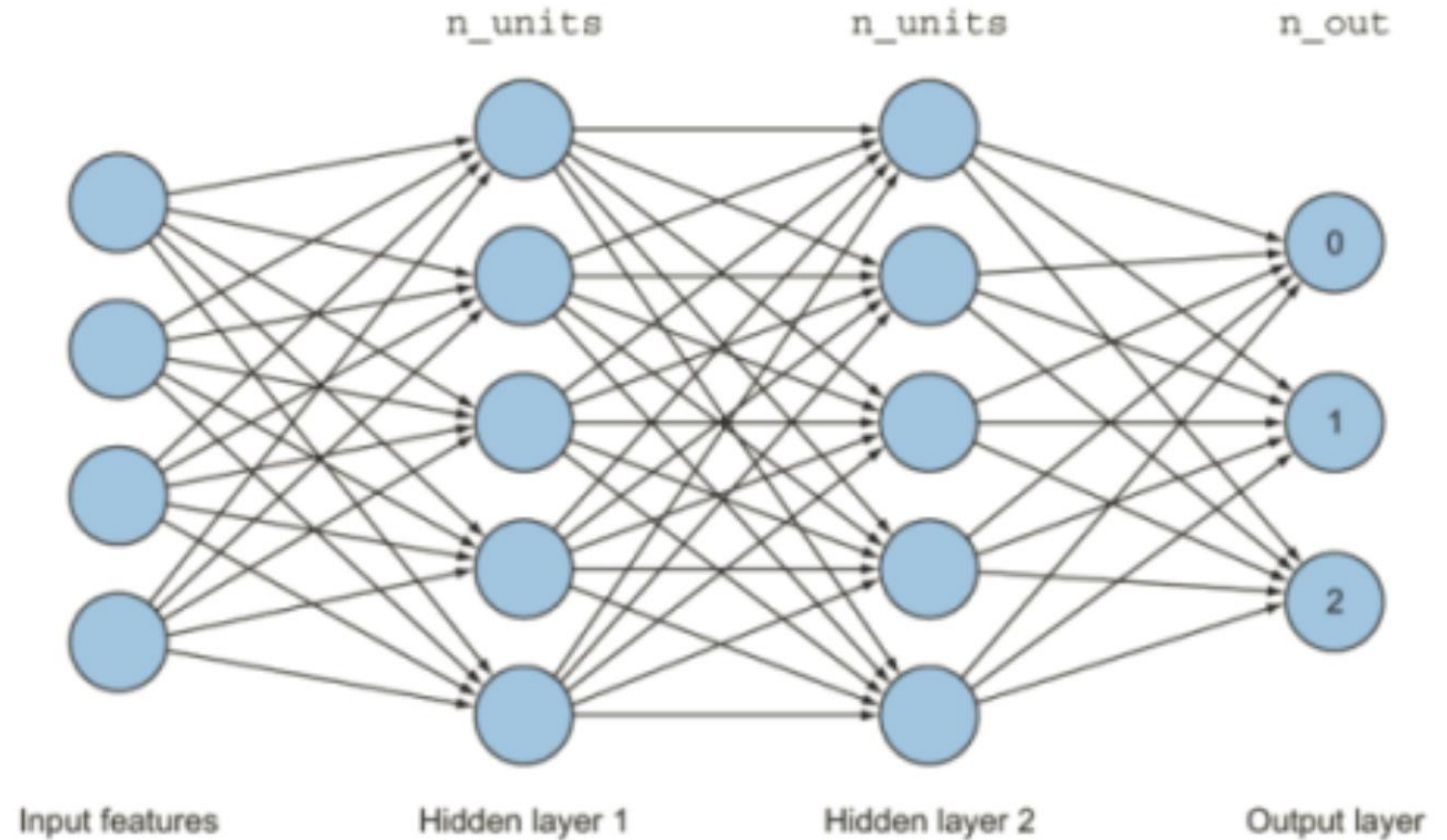
Weighted edges

Hidden Layers

Output Layer

Fully Connected Layers

- Total number of edges in the network
 - Weights_0_1:
(4 nodes in the input layer) $\times$ (5 nodes in layer 1)
+ 5 biases [1 bias per neuron]
= 25 edges
 - Weights_1_2:
5 $\times$ 5 nodes + 5 biases = 30 edges
 - Weights_2_output:
5 $\times$ 3 nodes + 3 bias = 18 edges
 - Total edges (weights) in this network = 73
- #weights = total number of trainable parameters in the network



Hyper Parameters

- Number of hidden layers
- Activation function
- Error Function
- Optimizer
- Batch size
- Number of epochs
- Learning rate



Reference

- Book – Deep Learning for Vision Systems by Mohamad Elgendy, Manning Publication, November 2020